Visitas Focadas

Atividade prática

Controlador de semáforo

cadence®

Do Arduino ao fluxo profissional de simulação e debug

Arduino • C++ • SystemVerilog • Xcelium • Verisium Debug

Sumário

Atividade prática – Visitas Focadas 2026/2 3
Objetivo da atividade 3
Descrição Geral do Problema 3
Regras do sistema: 4
Diagrama da Máquina de Estados 4
Parte 1 — Construindo o semáforo com Arduino 4
Protótipo com Arduino 4
Objetivo 4
Componentes: 4
Passos 5
Configuração do ambiente 5
Implementação do código 5
Parte 2 — Entendendo o circuito através do schematic 5
Validação do projeto: Schematic 5
Objetivo 5
Passos 5
Parte 3 — Validando o comportamento através do waveform 6
Validação do projeto: Waveform 6
Objetivo 6
Novos conceitos 6
Passos 6
Agora investigue o waveform: 7
Parte 4 — Conhecendo um fluxo profissional de simulação e debug 8
Aplicação do design nas ferramentas de simulação da Cadence 8
Objetivo 8
Passos 8
Conectando com o mundo real 8

Atividade prática – Visitas Focadas 2026/2

Objetivo da atividade

Ao longo da atividade, você irá: (1) montar e programar o controlador no Arduino; (2) relacionar a descrição em SystemVerilog ao circuito esquemático; (3) validar estados e transições por meio de formas de onda; e (4) repetir a análise com Xcelium e Verisium Debug em um fluxo profissional.

Nesta atividade, você irá construir e validar um controlador de semáforo em diferentes níveis de abstração. Primeiro, implementará o comportamento em C++ com Arduino; depois, relacionará esse comportamento a uma descrição em SystemVerilog; por fim, utilizará ferramentas de simulação e debug da Cadence para analisar o circuito. Ao concluir, deverá ser capaz de explicar a máquina de estados, verificar suas transições e identificar nos waveforms se as regras de segurança foram atendidas.

Descrição Geral do Problema

O controlador representa um cruzamento com fluxo de veículos e travessia de pedestres. Além de alternar as luzes em uma sequência previsível, o sistema deve responder ao pedido de travessia feito durante o sinal verde dos carros, sem permitir que os sinais verdes de carros e pedestres fiquem ativos simultaneamente.

Imagine um cruzamento movimentado em uma avenida principal. Durante o dia, o fluxo de veículos é intenso, mas também há grande circulação de pedestres. Para evitar acidentes e garantir acessibilidade, é necessário um sistema que contenha:

- **Três luzes para veículos:**
  - **Verde:** Permite a passagem dos carros.
  - **Amarela:** Indica atenção e prepara para a parada.
  - **Vermelha:** Proíbe a passagem dos carros.
- **Duas luzes para pedestres:**
  - **Verde:** Permite a travessia segura.
  - **Vermelha:** Proíbe a travessia.
- **Um botão de pedestre:**
  Quando pressionado durante o sinal verde para carros, deve interromper o ciclo normal e forçar a transição imediata para o amarelo, garantindo que o pedestre não espere indefinidamente.

Esse sistema deve ser implementado como uma **máquina de estados finitos (FSM)**, garantindo previsibilidade e segurança.

CAR_RED → CAR_GREEN → CAR_YELLOW → CAR_RED

A FSM terá três estados principais:

1. **CAR_RED (Estado inicial):**
   - Carros: Vermelho
   - Pedestres: Verde
   - Duração de 3 segundos*
2. **CAR_GREEN:**
   - Carros: Verde
   - Pedestres: Vermelho
   - Duração de até 3 segundos*, podendo ser antecipada pelo acionamento do botão.
3. **CAR_YELLOW:**
   - Carros: Amarelo

- Pedestres: Vermelho
- Duração de 2 segundos*

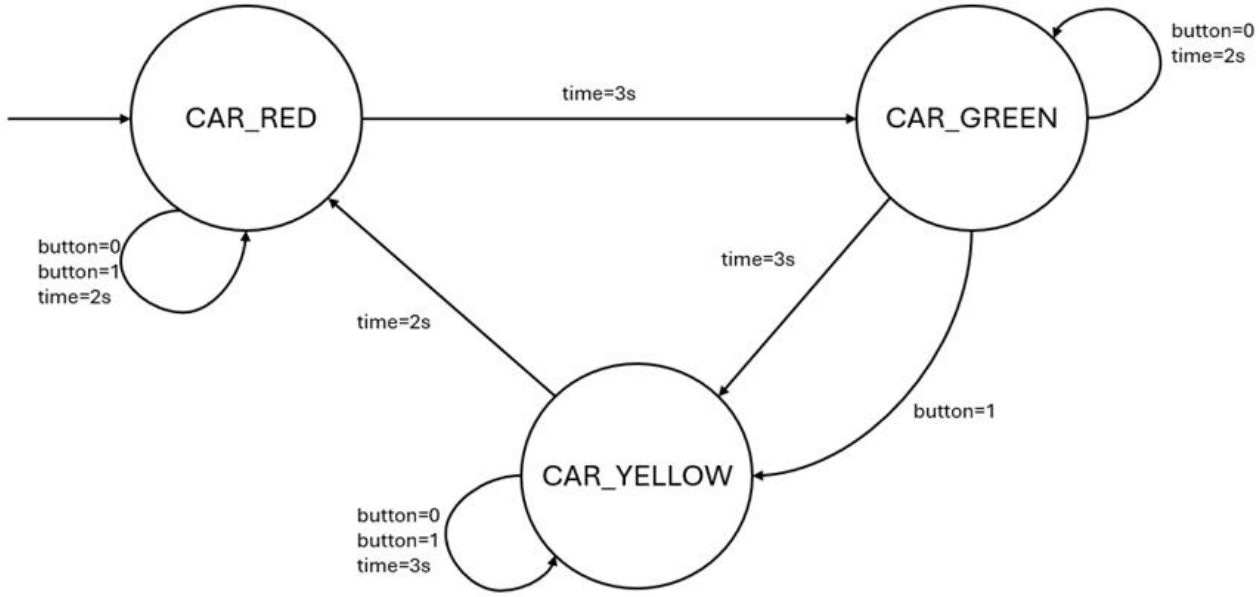
*Cada estado tem um tempo hipotético configurado para fins de simulação.

Regras do sistema:

- O botão só tem efeito quando os carros estão no verde;
- Após o amarelo, o sistema retorna ao vermelho, garantindo a travessia segura;

Os sinais verdes para carros e pedestres nunca devem permanecer ativos simultaneamente.

Diagrama da Máquina de Estados



Estado atual	Próximo estado				Saídas (LEDs)				
	time=2s	time=3s	button=0	button=1	carRed	carYellow	carGreen	pedestrianRed	pedestrianGreen
CAR_RED	CAR_RED	CAR_GREEN	CAR_RED	CAR_RED	1	0	0	0	1
CAR_YELLOW	CAR_RED	CAR_YELLOW	CAR_YELLOW	CAR_YELLOW	0	1	0	1	0
CAR_GREEN	CAR_GREEN	CAR_YELLOW	CAR_GREEN	CAR_YELLOW	0	0	1	1	0

Parte 1 — Construindo o semáforo com Arduino

Protótipo com Arduino

Objetivo

Criar um circuito funcional com LEDs representando as luzes do semáforo e um botão para pedestres.

Componentes:

- Arduino UNO
- 5 LEDs (3 para carros, 2 para pedestres)
- Resistores
- Botão

Passos

Configuração do ambiente

1. Acesse a ferramenta do Tinkercad e clique em “Inscrever-se” no canto superior direito;
2. Acesse o Traffic Light Project e clique em “Copiar e Usar” para duplicar o projeto;
3. Complete a montagem do circuito conforme o projeto no Tinkercad;
4. Clique em “Código” no canto superior direito para visualizar o código que define o comportamento do controlador.
    a. O código do Arduino sempre contém 2 funções:
        i. *void setup():* configura as portas;
        ii. *void loop():* executa continuamente e define o comportamento das portas configuradas.

Implementação do código

1. Complete a implementação do código em C++;
2. Inclua a lógica da máquina de estados com temporização e o tratamento do acionamento do botão para realizar a transição para o estado CAR_YELLOW;
3. Teste a simulação:
    a. Clique em “Iniciar Simulação” no canto superior direito para verificar o funcionamento;
    b. Observe:
        i. O estado inicial deve mostrar CAR_RED ativo;
        ii. Compare as transições com o diagrama da máquina de estados fornecido neste documento.

Parte 2 — Entendendo o circuito através do schematic

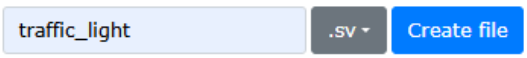
Validação do projeto: Schematic

Objetivo

Visualizar como a descrição em SystemVerilog é representada como lógica digital por meio de um diagrama esquemático.

Passos

1. Acesse a ferramenta do DigitalJS;
2. Crie um arquivo SystemVerilog:
    a. No campo “file name” digite: “traffic_light”
    b. Selecione “.sv”
    c. Clique em “Create File”



3. Insira o design do projeto:
    a. Copie o código presente no link e cole na ferramenta.
4. Execute a análise do projeto:
    a. Clique no botão “Run”, localizado no canto superior direito da tela.
5. Visualize e valide o diagrama:
    a. Após rodar, o esquemático será gerado automaticamente;
    b. Clique no botão “rst” para iniciar a execução;
    c. Observe:
        i. A lógica das saídas (carRed, carYellow, carGreen);

ii. Como os elementos do circuito estão conectados para implementar a lógica do controlador.

Parte 3 — Validando o comportamento através do waveform

Validação do projeto: Waveform

Objetivo

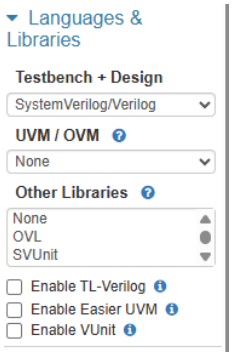
Simular a implementação da máquina de estados em SystemVerilog e analisar seu comportamento por meio das formas de onda.

Novos conceitos

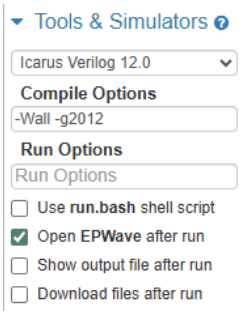
Design: contém a implementação do circuito que será testado; neste projeto, corresponde ao controlador do semáforo. Testbench: funciona como o ambiente de teste do design, gerando entradas como clock, reset e acionamento do botão. A simulação combina os dois: o testbench aplica os estímulos, e o design responde por meio dos estados e sinais de saída. Waveform: representação visual da mudança dos sinais digitais ao longo do tempo. Neste projeto, ele deve permitir confirmar o estado inicial, a duração de cada estado, a resposta ao botão de pedestres, a passagem obrigatória por CAR_YELLOW e a impossibilidade de carGreen e pedestrianGreen permanecerem ativos simultaneamente.

Passos

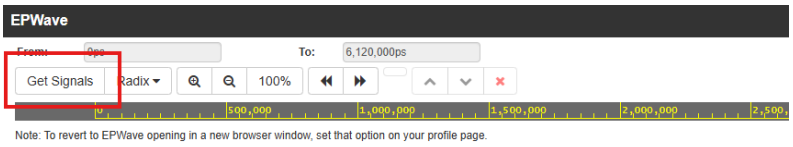
1. Acesse a ferramenta do EDA Playground e faça login na ferramenta;
2. Acesse o projeto Traffic Light Verilog e verifique se:
   a. Na barra de ferramentas à esquerda da tela, no tópico “Languages & Libraries” está selecionada a opção “SystemVerilog/Verilog”:



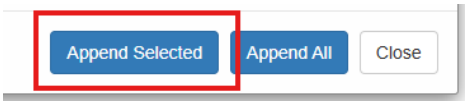
   b. Na barra de ferramentas à esquerda da tela, no tópico “Tools & Simulators” está selecionada a opção “Icarus Verilog 12.0” e está marcada a opção “Open EPWave after run”:



3. Use o testbench e o design fornecidos para realizar a simulação;
4. Execute a simulação:
    a. Clique em "Run", no canto superior esquerdo, para executar a simulação e abrir o waveform.
5. Configure os sinais no waveform:
    a. Na tela do waveform, clique em "Get Signals":



    b. Em seguida, clique em ".dut":
    c. Selecione os sinais(utilize ctrl para selecionar vários sinais):
        i. button
        ii. carRed
        iii. carYellow
        iv. carGreen
        iv. pedestrianRed
        v. pedestrianGreen
    d. Clique em "Append Selected" para adicioná-los:



6. Visualize e valide a lógica:
    a. Analise o waveform para confirmar o funcionamento;
    Dica: *Clicando na régua superior, é possível ver os valores dos sinais em pontos específicos;*

    b. Valide:
        i. Se o estado inicial mostra CAR_RED ativo;
        ii. Se as transições condizem com a tabela de transições abaixo do diagrama da máquina de estados fornecido neste documento;
        iii. O tempo de cada transição, considerando o clique do botão de pedestres.

Agora investigue o waveform:

- Qual é o estado do semáforo logo após o reset?
- Quanto tempo o sinal permanece em CAR_RED?
- O que acontece quando button = 1 durante CAR_GREEN?
- O CAR_YELLOW ocorre antes de retornar a CAR_RED?

- Os sinais carGreen e pedestrianGreen ficam ativos simultaneamente em algum momento? Isso deveria acontecer? Por quê?

Parte 4 — Conhecendo um fluxo profissional de simulação e debug

Aplicação do design nas ferramentas de simulação da Cadence

Objetivo

Executar o mesmo design utilizando ferramentas profissionais da Cadence e entender como os conceitos vistos anteriormente aparecem em um fluxo real de desenvolvimento e verificação de circuitos digitais.

Até este momento, utilizamos ferramentas educacionais para construir, simular e analisar o controlador do semáforo. Agora, utilizaremos ferramentas empregadas profissionalmente no desenvolvimento de sistemas eletrônicos.

Passos

1. Execute o design e o testbench utilizando o Xcelium.
2. Abra os resultados no Verisium Debug.
3. Analise os sinais no Waveform.
4. Adicione uma expressão para auxiliar a análise dos sinais.
5. Abra o Schematic e localize a lógica responsável pelos sinais do semáforo.
6. Compare o que você observa no Verisium Debug com o que foi analisado anteriormente no EPWave e no DigitalJS.

Conectando com o mundo real

O controlador de semáforo é um exemplo simplificado de circuito digital. Em projetos reais, os circuitos podem ser muito maiores e mais complexos; por isso, ferramentas de simulação e debug são essenciais para descrever o comportamento esperado, executar testes, analisar waveforms e localizar a lógica responsável por eventuais falhas.

O mesmo processo realizado durante esta atividade, descrever, simular, analisar e encontrar possíveis problemas, também faz parte do desenvolvimento e da verificação de sistemas digitais reais.